

QUESTION NO: 15

- Write pseudocode for a task receiving messages from a queue and signaling completion using event flags.

PROBLEM ANALYSIS:

The task acts as a consumer to receive messages from a queue and process them. After successful processing, it signals completion using an event flag so that other tasks can synchronize on it.

RTOS SELECTION:

FreeRTOS is chosen for this solution.

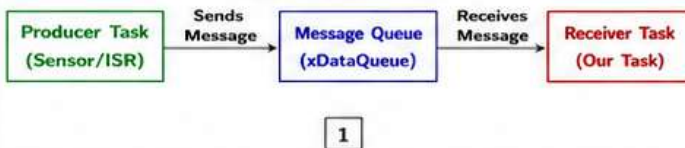
REQUIRED PRIMITIVES:

- Queue for inter-task communication (xQueueReceive)
- Event Group for signaling completion (xEventGroupSetBits)
- Blocking API to wait efficiently without wasting CPU cycles

DATA STRUCTURE:

- xDataQueue : QueueHandle_t
- xCompletionEventGroup : EventGroupHandle_t
- message_t : Data type for messages
- TASK_DONE_BIT : Event bit for completion

SYSTEM OVERVIEW



PSEUDOCODE (Contd...)

BEGIN

// SYSTEM INITIALIZATION

```
CREATE QUEUE xDataQueue WITH SIZE QUEUE_SIZE
CREATE EVENT GROUP xCompletionEventGroup
CREATE TASK ReceiverTask WITH PRIORITY N
START SCHEDULER
```

END

TASK: ReceiverTask

BEGIN

```
DECLARE message_t msg
DECLARE status return_status
INITIALIZE timeout_val = WAIT_TIME

WHILE TRUE
    RECEIVE MESSAGE FROM QUEUE
    status = xQueueReceive(xDataQueue, &msg, timeout_val)
    IF status == PASS THEN
        PROCESS THE RECEIVED MESSAGE
        ProcessMessage(msg)
        SIGNAL COMPLETION USING EVENT FLAG
        xEventGroupSetBits(xCompletionEventGroup, TASK_DONE_BIT)
    ELSE
        // Timeout occurred, no message received
        CONTINUE // Go back and wait again
    END IF
END WHILE
END
```

2

PSEUDOCODE (Contd...)

TASK: WaitingTask

BEGIN

```
DECLARE EventBits_t received_bits
DECLARE wait_flag = TASK_DONE_BIT
DECLARE wait_option = pdTRUE

WHILE TRUE
    // WAIT UNTIL COMPLETION FLAG IS SET
    received_bits = xEventGroupWaitBits(xCompletionEventGroup,
                                        wait_flag, pdTRUE, pdFALSE, portMAX_DELAY)
    IF (received_bits & TASK_DONE_BIT) != 0 THEN
        // Completion received
        PerformNextOperation()
    END IF
END WHILE
END
```

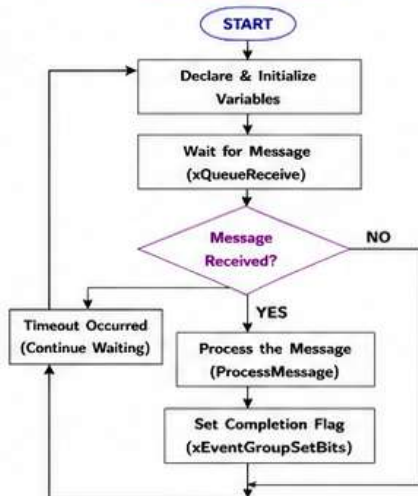
QUEUE OPERATION SUMMARY

Operation	Performed By	Purpose
xQueueSend(xDataQueue)	Producer Task	Place message into queue
xQueueReceive(xDataQueue)	Receiver Task	Retrieve message from queue
xEventGroupSetBits()	Receiver Task	Signal completion
xEventGroupWaitBits()	Waiting Task(s)	Wait for completion signal

3

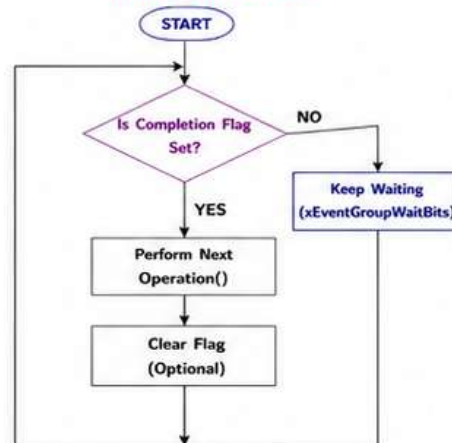
LOGICAL FLOW DIAGRAM

RECEIVER TASK FLOW



4

WAITING TASK FLOW



EXPLANATION:

The ReceiverTask waits for messages from the queue. When a message is received within the wait time, it is processed and the completion flag (TASK_DONE_BIT) is set in the event group. Other tasks waiting on this flag using xEventGroupWaitBits() will be notified and can proceed with their next operations.

KEY POINTS:

- Queues are used for data transfer between tasks.
- Event groups are used for status signaling and synchronization.
- The task blocks efficiently while waiting, avoiding CPU wastage.
- Multiple tasks can wait on the same event flag.

ASSUMPTIONS:

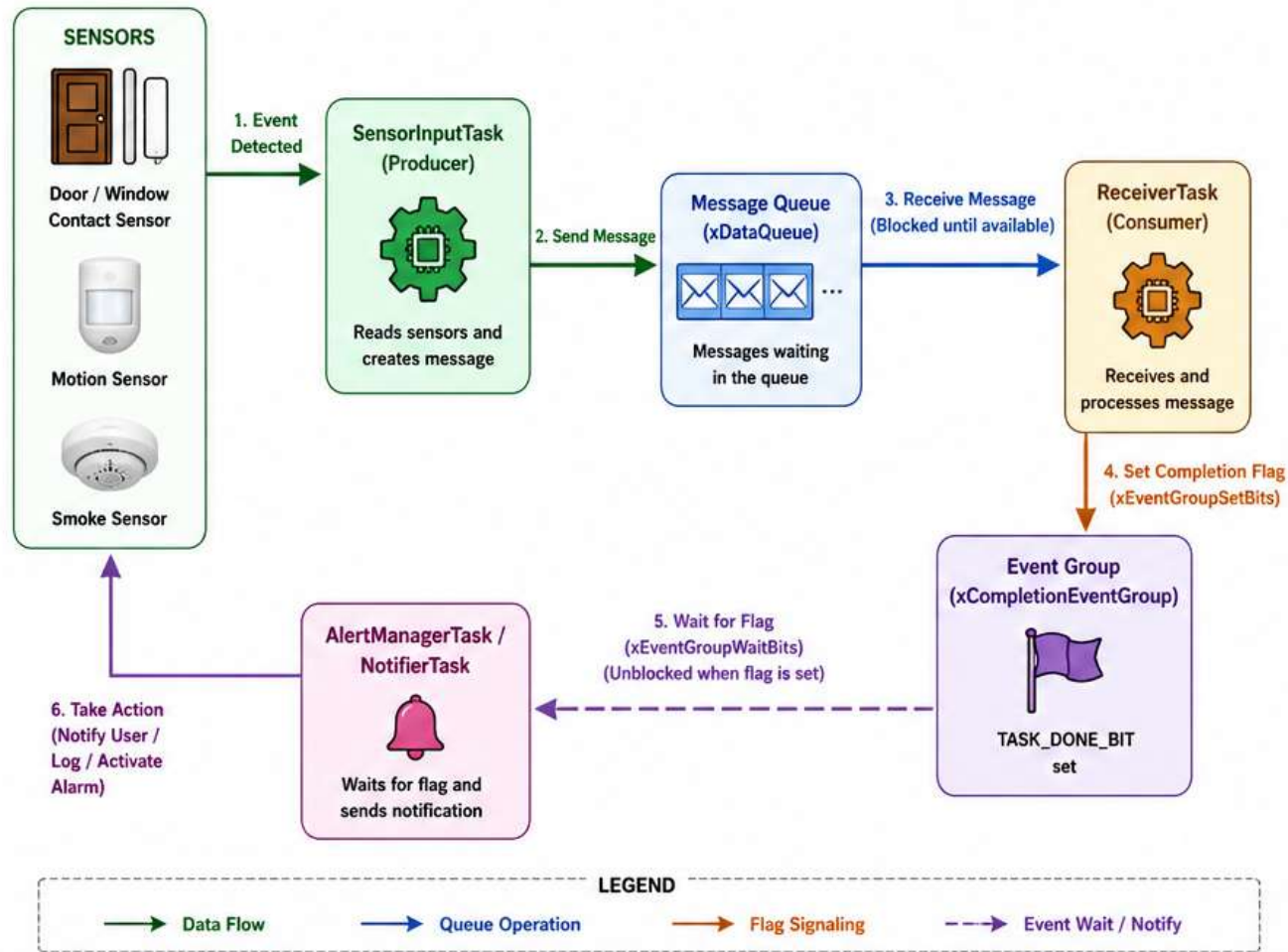
- Queue and event group are created during system initialization.
- Message processing is fast and non-blocking.
- WAIT_TIME is a predefined timeout value.
- Only one completion bit (TASK_DONE_BIT) is used for simplicity.
- No priority inversion issues occur.

5

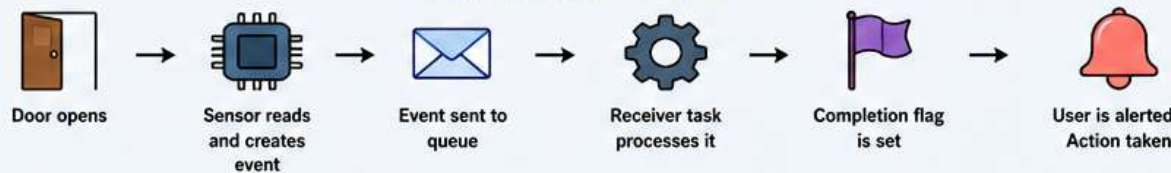
7. REAL-LIFE EXAMPLE

Example: A smart home security gateway.

SMART HOME SECURITY GATEWAY – FLOW OF OPERATIONS



WHAT HAPPENS IN REAL LIFE?



REAL-LIFE EXAMPLE:

Example: A smart home security gateway.

Consider a Wi-Fi enabled home security gateway built on an embedded RTOS. A “SensorInputTask” (producer) collects readings from door/window contact sensors and pushes each detected event (e.g., “door opened”) into a message queue (xDataQueue).

A “ReceiverTask” (the task described in the pseudocode) blocks on xQueueReceive(). When it gets a message, it processes the event — for example, it might log it to local storage, check whether it violates a security rule (e.g., door opened while system is armed), and take appropriate action.

After processing, it calls xEventGroupSetBits(xCompletionEventGroup, TASK_DONE_BIT) to signal completion. An “AlertManagerTask”, which previously called xEventGroupWaitBits() to wait for this flag, is immediately unblocked and can now send a push notification to the homeowner’s phone and activate a local alarm.

HOW THIS SOLVES THE PROBLEM:

- The queue ensures reliable, ordered delivery of sensor events without busy-waiting.
- The event flag provides efficient, one-to-many completion notification without requiring polling.
- The system remains responsive and power-efficient — ideal for a battery-backed embedded gateway.

BENEFITS IN REAL LIFE:

- Ensures no sensor event is missed.
- Keeps CPU usage low (tasks block instead of polling).
- Fast and reliable notification to other tasks.
- Scalable: more tasks can wait on the same event flag if needed.

QUESTION NO: 16

- Write pseudocode for a three-stage processing pipeline using queues and semaphores.

PROBLEM ANALYSIS:

The question describes a data processing pipeline made of three independent stages, where the output of one stage becomes the input of the next. Two distinct RTOS requirements are embedded in this:

1. Inter-task communication across stages: each stage runs as a separate task, and data must move from Stage 1 → Stage 2 → Stage 3 reliably, in order, without the sender and receiver needing to run at the same instant. This calls for queues.
2. Synchronization / mutual exclusion within a stage: one of the stages (Stage 2 in this solution) accesses a shared resource that must not be used by more than one context at a time. This calls for a semaphore.

RTOS SELECTION:

FreeRTOS is selected for this solution.

- FreeRTOS provides ready-made queue objects (xQueueSend / xQueueReceive) that are ideal for connecting pipeline stages.
- FreeRTOS binary semaphores (xSemaphoreTake / xSemaphoreGive) give a lightweight mutual-exclusion mechanism.
- FreeRTOS's preemptive priority scheduler lets all three stages execute concurrently and efficiently on a single core.

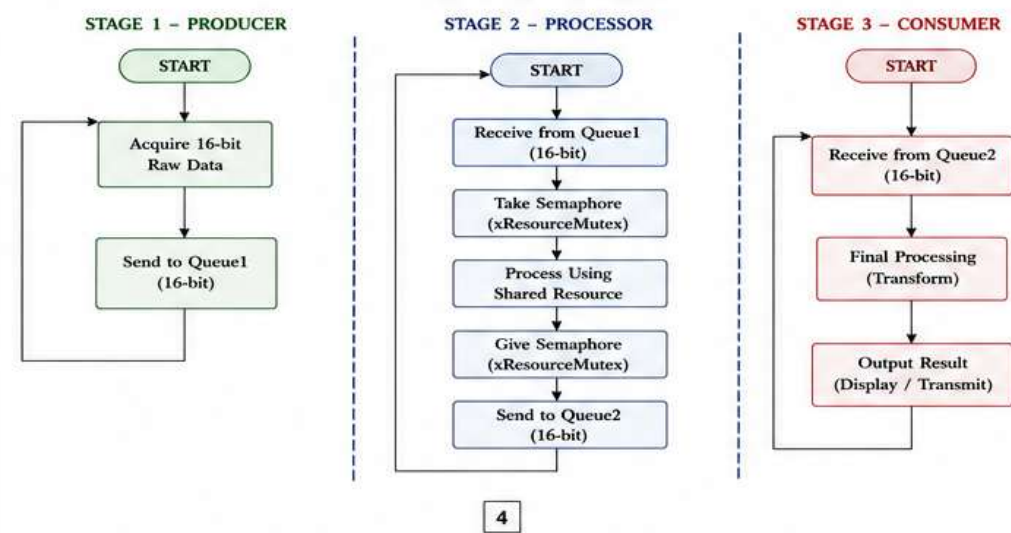
DATA STRUCTURE (Handles):

Queue1	: QueueHandle_t	(Stage 1 → Stage 2)
Queue2	: QueueHandle_t	(Stage 2 → Stage 3)
xResourceMutex	: SemaphoreHandle_t	(Binary semaphore)

SYSTEM OVERVIEW:



LOGICAL FLOW DIAGRAM



PSEUDOCODE (Contd...)

```
// Global handles (created during system initialization)
QueueHandle_t Queue1; // Stage 1 → Stage 2
QueueHandle_t Queue2; // Stage 2 → Stage 3
SemaphoreHandle_t xResourceMutex; // protects shared resource in Stage 2
#define DATA_BIT_WIDTH 16 // 16-bit data format
typedef uint16_t data16_t; // 16-bit data type
```

```
TASK Stage1_Producer(void *pvParameters)
BEGIN
    data16_t data;
    LOOP FOREVER
        data = AcquireRawData(); // e.g., read 16-bit sensor value
        xQueueSend(Queue1, &data, portMAX_DELAY);
    END LOOP
END TASK
```

```
TASK Stage2_Processor(void *pvParameters)
BEGIN
    data16_t data, processed;
    LOOP FOREVER
        status = xQueueReceive(Queue1, &data, portMAX_DELAY);
        IF status == PASS THEN
            xSemaphoreTake(xResourceMutex, portMAX_DELAY);
            processed = ProcessStage2(data); // uses shared resource
            xSemaphoreGive(xResourceMutex);
            xQueueSend(Queue2, &processed, portMAX_DELAY);
        END IF
    END LOOP
END TASK
```

2

PSEUDOCODE (Contd...)

```
TASK Stage3_Consumer(void *pvParameters)
BEGIN
    data16_t processed, result;
    LOOP FOREVER
        status = xQueueReceive(Queue2, &processed, portMAX_DELAY);
        IF status == PASS THEN
            result = ProcessStage3(processed); // final transform
            OutputResult(result); // e.g., display / transmit
        END IF
    END LOOP
END TASK
```

QUEUE OPERATIONS SUMMARY:

Operation	Performed By	Purpose
xQueueSend(Queue1, &data)	Stage 1	Send 16-bit data to Stage 2
xQueueReceive(Queue1, &data)	Stage 2	Receive data from Stage 1
xQueueSend(Queue2, &processed)	Stage 2	Send processed 16-bit data to Stage 3
xQueueReceive(Queue2, &processed)	Stage 3	Receive data from Stage 2

SEMAPHORE USAGE:

Semaphore	Used By	Purpose
xResourceMutex (Binary)	Stage 2	Ensure exclusive access to shared resource

3

EXPLANATION:

Three tasks form a pipeline. Stage 1 produces 16-bit data and sends it to Stage 2 via Queue1. Stage 2 receives the data, uses a semaphore to access a shared resource safely, processes it, and sends the 16-bit result to Stage 3 via Queue2. Stage 3 receives the data and outputs the final result.

KEY POINTS:

- Queues provide ordered, reliable data transfer between stages.
- Semaphore ensures mutual exclusion for the shared resource in Stage 2.
- All tasks block (not poll) when waiting, saving CPU time.
- 16-bit data format is used across the pipeline.

ASSUMPTIONS:

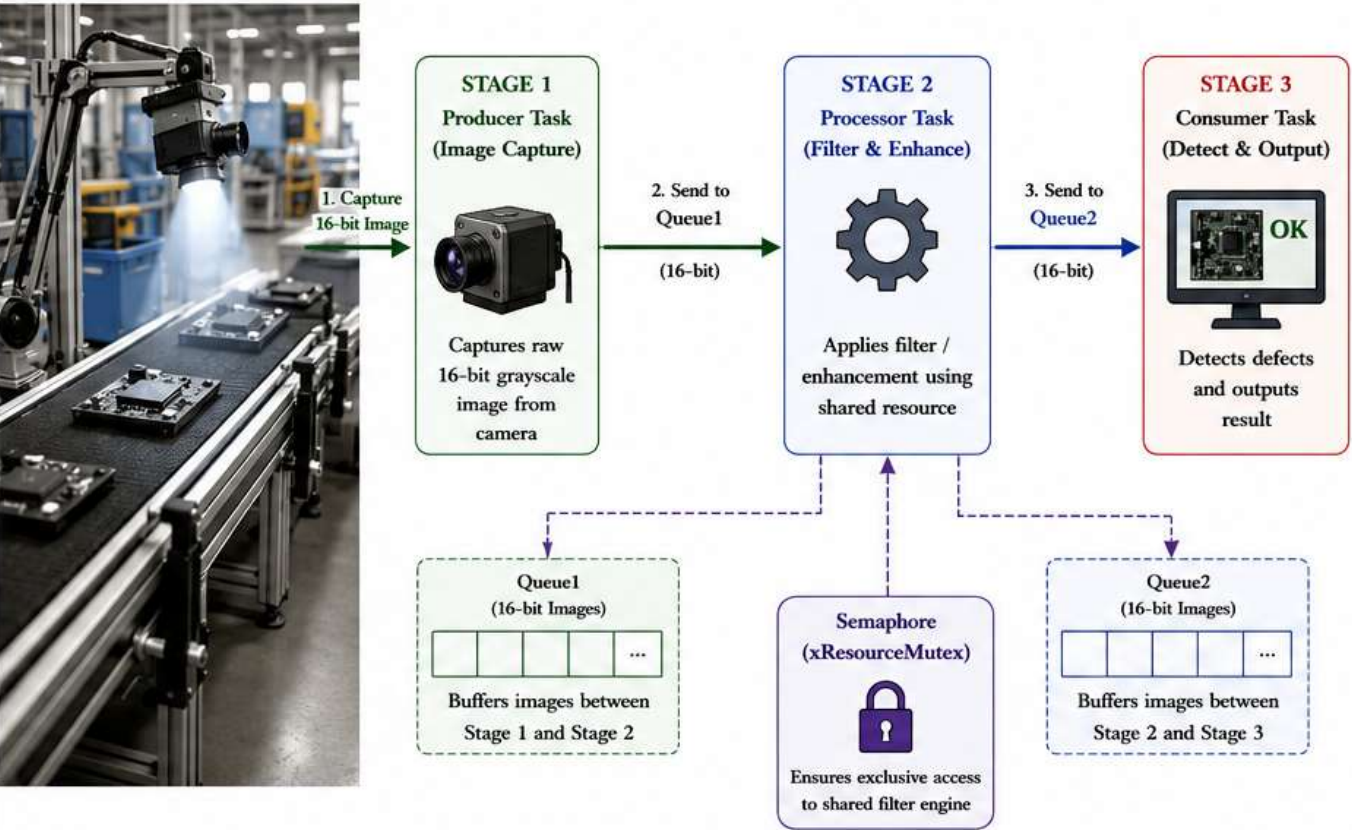
- Queues are created with sufficient length for 16-bit data items.
- xResourceMutex is a binary semaphore created in the given state.
- Processing functions are fast and non-blocking.
- All tasks run at priorities managed by the RTOS scheduler.

5

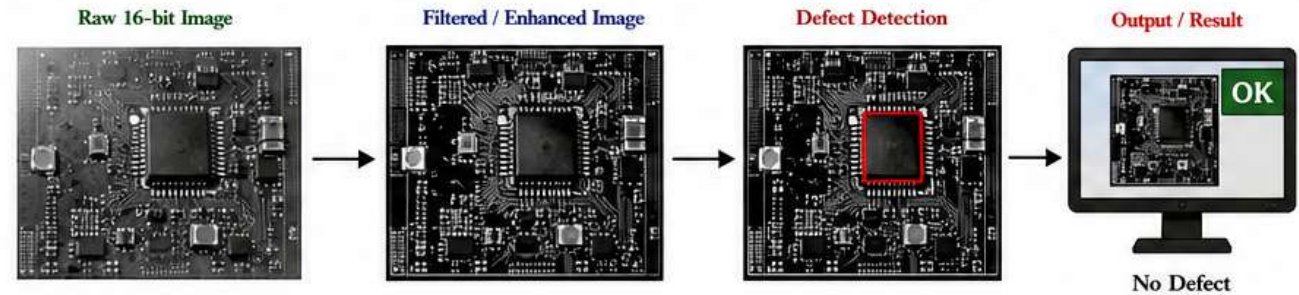
7. REAL-LIFE EXAMPLE

Example: Real-time Image Processing Pipeline in an Industrial Inspection System

REAL-TIME IMAGE PROCESSING PIPELINE – FLOW OF OPERATIONS



EXAMPLE FLOW IN REAL TIME



REAL-LIFE EXAMPLE:

Example: Real-time Image Processing Pipeline in an Industrial Inspection System

Consider an automated quality inspection system on a production line. A high-speed camera captures 16-bit grayscale images of each product (for example, a printed circuit board).

Stage 1 (Image Capture Task) acquires a 16-bit image and places it into Queue1. Stage 2 (Processing Task) receives the image, takes a semaphore to access a shared hardware filter engine (a special image processing accelerator), performs filtering and enhancement, and then sends the processed 16-bit image to Queue2. Stage 3 (Detection & Output Task) receives the image, detects any defects, and displays or transmits the final result.

This pipeline ensures that image capture, processing, and decision output happen concurrently and efficiently without data loss, while the semaphore guarantees that the expensive shared filter engine is used safely by only one processing task at a time.

HOW THIS SOLVES THE PROBLEM:

- Queues provide reliable, ordered transfer of 16-bit images between stages.
- Semaphore protects the shared processing resource in Stage 2.
- Each task blocks when waiting, avoiding CPU wastage.
- The pipeline increases throughput by allowing all three stages to work in parallel.

BENEFITS IN REAL LIFE:

- High-speed inspection with real-time response.
- No image loss even if one stage is slower.
- Efficient CPU and resource utilization.
- Scalable to higher resolutions and more stages.

ASSUMPTIONS:

- Queues are created with enough length for 16-bit images.
- xResourceMutex is a binary semaphore created in the given state.
- Image processing functions are fast and non-blocking.
- Tasks are scheduled with appropriate priorities by the RTOS.

QUESTION NO: 17

- Write pseudocode for periodic and event tasks posting messages to a priority mailbox.

PROBLEM ANALYSIS:

The question combines three RTOS concepts that the scheduler must coordinate at run time:

- A periodic task that must run at a fixed, repeating time interval regardless of what else is happening in the system.
- An event task that must run only when an external event occurs (e.g., an interrupt), and must respond with minimal delay.
- A shared "priority mailbox" that both tasks post messages into, where urgent (event) messages must be read before older routine (periodic) messages.

The core problem is how the scheduler keeps a periodic task on time, immediately switches to an event task when triggered, and ensures messages are handled in priority order in a shared mailbox.

RTOS SELECTION:

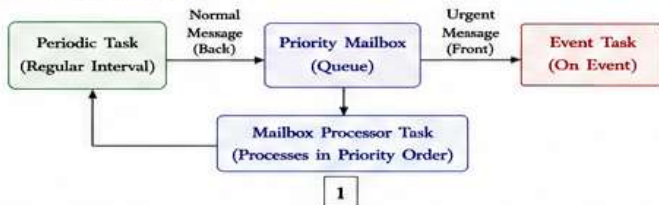
FreeRTOS is selected for this solution.

- Preemptive priority scheduler ensures event task preempts all others.
- `vTaskDelayUntil()` provides jitter-free periodic execution.
- Queue can behave as a priority mailbox using send-to-front/back.
- `xQueueGiveFromISR()` allows immediate handoff from ISR.

DATA STRUCTURE (HANDLES):

xMailbox	: QueueHandle_t	(Priority mailbox)
xEventSem	: SemaphoreHandle_t	(Binary semaphore)

SYSTEM OVERVIEW:



PSEUDOCODE (Contd...)

```
// Global handles (created during system initialization)
QueueHandle_t    xMailbox;    // shared priority mailbox
SemaphoreHandle_t xEventSem;   // given by ISR, taken by EventTask
```

```
-----
TASK PeriodicTask(void *pvParameters)    // priority = Medium
BEGIN
    xLastWakeTime = GetCurrentTick();
    LOOP FOREVER
        // Scheduler suspends this task until next period tick
        vTaskDelayUntil(&xLastWakeTime, PERIOD_TICKS);
        msg = BuildRoutineMessage();
        xQueueSendToBack(xMailbox, &msg, 0); // normal priority
    END LOOP
END TASK
-----
```

```
-----
TASK EventTask(void *pvParameters)    // priority = High
BEGIN
    LOOP FOREVER
        // Scheduler blocks this task until ISR gives xEventSem
        xSemaphoreTake(xEventSem, portMAX_DELAY);
        msg = BuildUrgentMessage();
        xQueueSendToFront(xMailbox, &msg, 0); // high priority
    END LOOP
END TASK
-----
```

```
-----
ISR ExternalEventHandler()
BEGIN
    xSemaphoreGiveFromISR(xEventSem, &xHigherPriorityTaskWoken);
    // Ask scheduler to context-switch immediately if
    // EventTask now has higher priority
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
END ISR
-----
```

PSEUDOCODE (Contd...)

```
TASK MailboxProcessorTask(void *pvParameters)    // priority = Low
BEGIN
    LOOP FOREVER
        // Scheduler blocks this task until any message is available
        status = xQueueReceive(xMailbox, &msg, portMAX_DELAY);
        IF status == PASS THEN
            ProcessMessage(msg); // urgent items handled first
        END IF
    END LOOP
END TASK
-----
```

QUEUE BEHAVIOR (PRIORITY MAILBOX):

Operation	Function Used	Priority	Purpose
Post routine message (Periodic)	<code>xQueueSendToBack()</code>	Normal	Added to end of queue
Post urgent message (Event)	<code>xQueueSendToFront()</code>	High	Added to front of queue
Receive message (Processor)	<code>xQueueReceive()</code>	Front first	Handles high priority first

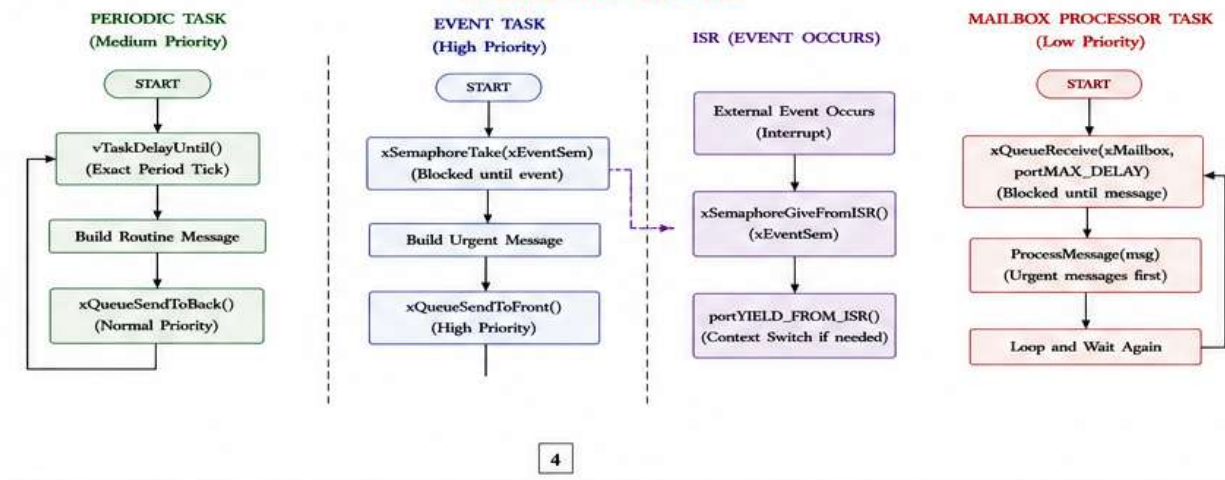
SEMAPHORE USAGE:

Semaphore	Given By	Taken By	Purpose
xEventSem (Binary)	ISR (Event)	EventTask	Signals event task to run

TASK PRIORITIES:

Task	Priority Level
EventTask	High (Preempts others)
PeriodicTask	Medium
MailboxProcessorTask	Low (Background processing)

LOGICAL FLOW DIAGRAM



EXPLANATION:

The periodic task posts routine messages at fixed intervals using `xQueueSendToBack()`. The event task posts urgent messages using `xQueueSendToFront()` when an event occurs. The mailbox processor task always receives from the front of the queue, ensuring urgent messages are handled first.

KEY POINTS:

- Preemptive scheduler ensures event task runs immediately when triggered.
- Priority mailbox ensures urgent messages are processed before routine ones.
- `vTaskDelayUntil()` provides accurate periodic execution without drift.
- ISR signals event task and requests immediate context switch.
- Mailbox processor runs in background with lowest priority.

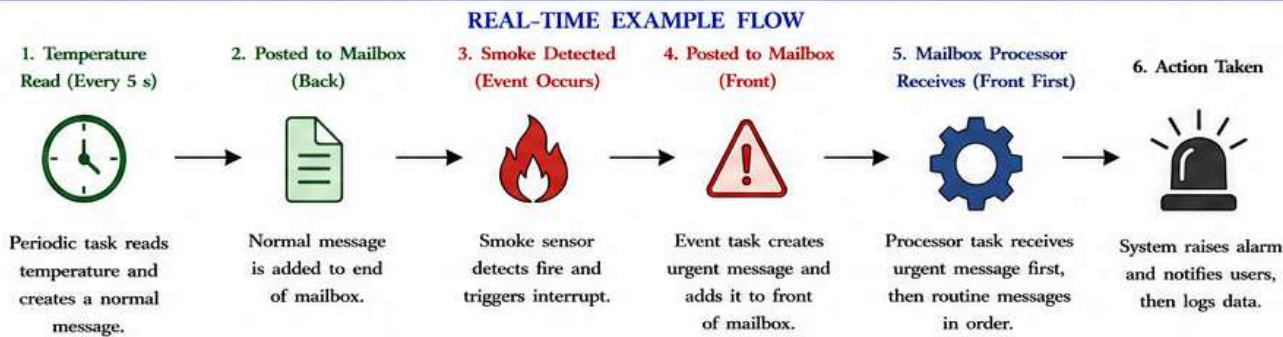
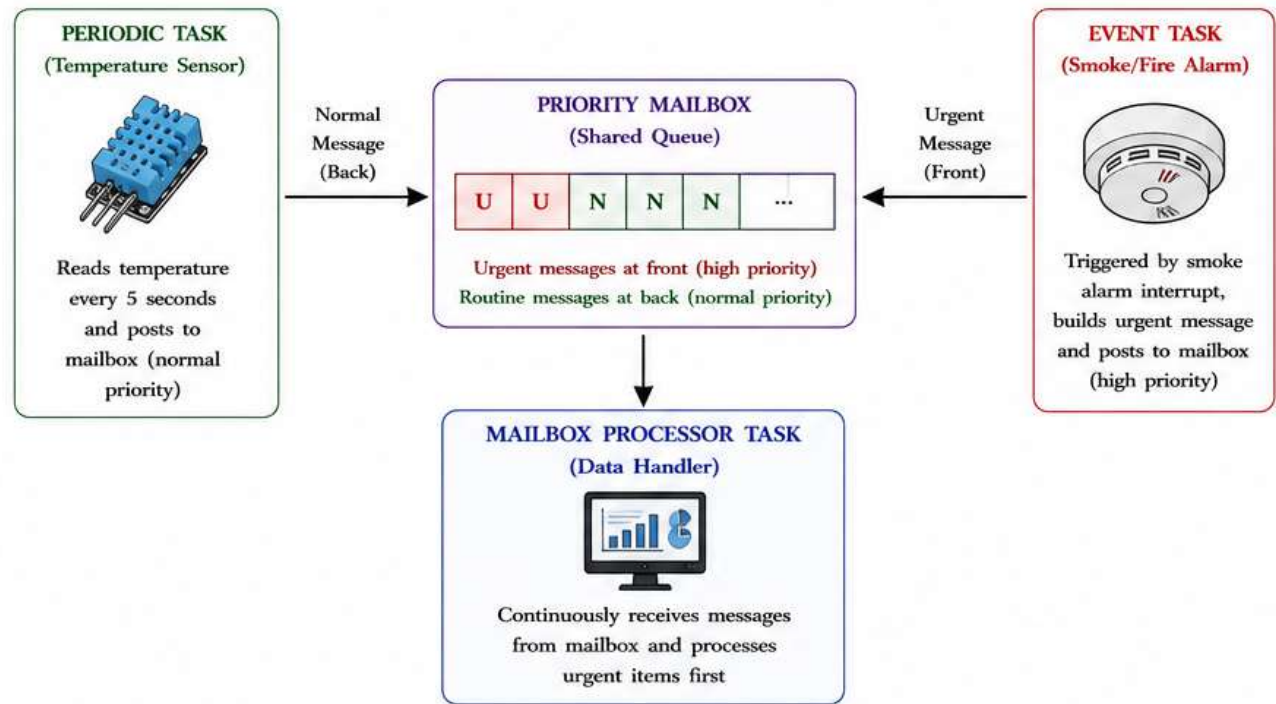
ASSUMPTIONS:

- System tick is 1 ms (16-bit tick count used).
 - `xEventSem` is a binary semaphore created in the given state.
 - Queue length is sufficient to hold pending messages.
 - `ProcessMessage()` is fast and non-blocking.
 - EventTask has higher priority than other tasks.
- 5

7. REAL-LIFE EXAMPLE

Example: Smart Building Environmental Monitoring and Alarm System

REAL-TIME ENVIRONMENTAL MONITORING SYSTEM – FLOW OF OPERATION



SCENARIO:

In a smart building, temperature is monitored every 5 seconds by a periodic task. When smoke is detected, an interrupt triggers the event task, which immediately posts an urgent alarm message to the mailbox. The processor task handles the alarm first, ensuring fast response and safety.

REAL-LIFE EXAMPLE:

Consider a smart building environmental monitoring system.

A temperature sensor task reads the temperature every 5 seconds and posts the reading to a shared mailbox as a normal message.

A smoke detector is connected to a hardware interrupt. When smoke is detected, the interrupt triggers an event task that immediately posts an urgent alarm message to the same mailbox.

The mailbox processor task always handles urgent alarm messages before routine temperature updates, ensuring quick and reliable response to emergencies.

HOW THIS SOLVES THE PROBLEM:

- Periodic task runs at fixed interval without drift (vTaskDelayUntil).
- Event task responds immediately to external events (interrupt driven).
- Priority mailbox ensures urgent messages are processed before routine ones.
- Preemptive scheduler guarantees fast response to critical events.

BENEFITS IN REAL LIFE:

- Ensures building safety by handling alarms with zero delay.
- Accurate environmental monitoring at regular intervals.
- Prevents alarm messages from waiting behind routine data.
- Efficient and reliable operation using RTOS scheduling.

ASSUMPTIONS:

- System tick is 1 ms (16-bit tick count used).
- xEventSem is a binary semaphore created in the given state.
- Mailbox queue length is sufficient to hold pending messages.
- Processing of each message is fast and non-blocking.
- Event task has higher priority than other tasks.

QUESTION NO: 18

- Write pseudocode to trigger a motor control task every 50ms using a software timer.

PROBLEM ANALYSIS:

The question asks for a fixed 50 ms trigger that is generated by a software timer rather than by a task's own delay loop, and used to drive a motor control task.

- Software timer callbacks in FreeRTOS run inside the single Timer Service (Daemon) Task. If the callback did the full motor control work, it would block all other timers.
- Motor control logic needs its own task priority and can take variable time. Therefore, keep the timer callback short and signal a semaphore to wake a separate MotorControlTask.

RTOS SELECTION:

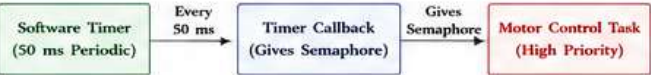
FreeRTOS is selected for this solution.

- Software timers (xTimerCreate/xTimerStart) work on the tick.
- Auto-reload (pdTRUE) makes the timer periodic.
- Binary semaphore in callback wakes the motor control task.
- Preemptive scheduler runs the motor control task with high priority.

DATA STRUCTURE (HANDLES):

xMotorTimer	: TimerHandle_t	(50 ms periodic timer)
xMotorSem	: SemaphoreHandle_t	(Binary semaphore)

SYSTEM OVERVIEW:



1

PSEUDOCODE (Contd...)

```
// Global handles (created during system initialization)
TimerHandle_t    xMotorTimer;
SemaphoreHandle_t xMotorSem;

// Timer callback - executes in the Timer Service Task context
// Kept short and non-blocking, as the scheduler runs ALL
// software timers through this one shared task

FUNCTION MotorTimerCallback(TimerHandle_t xTimer)
BEGIN
    xSemaphoreGive(xMotorSem);
END FUNCTION

TASK MotorControlTask(void *pvParameters)    // priority = High
BEGIN
    LOOP FOREVER
        // Scheduler blocks this task until the timer signals it
        xSemaphoreTake(xMotorSem, portMAX_DELAY);
        UpdateMotorControl();    // e.g., recompute PWM duty cycle
    END LOOP
END TASK

FUNCTION main()
BEGIN
    xMotorSem = xSemaphoreCreateBinary();
    xMotorTimer = xTimerCreate("MotorTimer",
        pdMS_TO_TICKS(50),    // 50 ms period
        pdTRUE,                // auto-reload = periodic
        NULL,
        MotorTimerCallback);
    CreateTask(MotorControlTask, PRIORITY_HIGH);
    xTimerStart(xMotorTimer, 0);
    vTaskStartScheduler();    // scheduler takes over from here
END FUNCTION
```

2

PSEUDOCODE (Contd...)

RTOS OBJECTS SUMMARY:

Object	Type	Created By	Used By	Purpose
xMotorTimer	Software Timer	main()	Timer Service Task	Generates 50 ms periodic trigger
xMotorSem	Binary Semaphore	main()	Callback & Motor Task	Signals motor task
MotorControlTask	Task	main()	Scheduler	Performs motor control work

TIMER DETAILS:

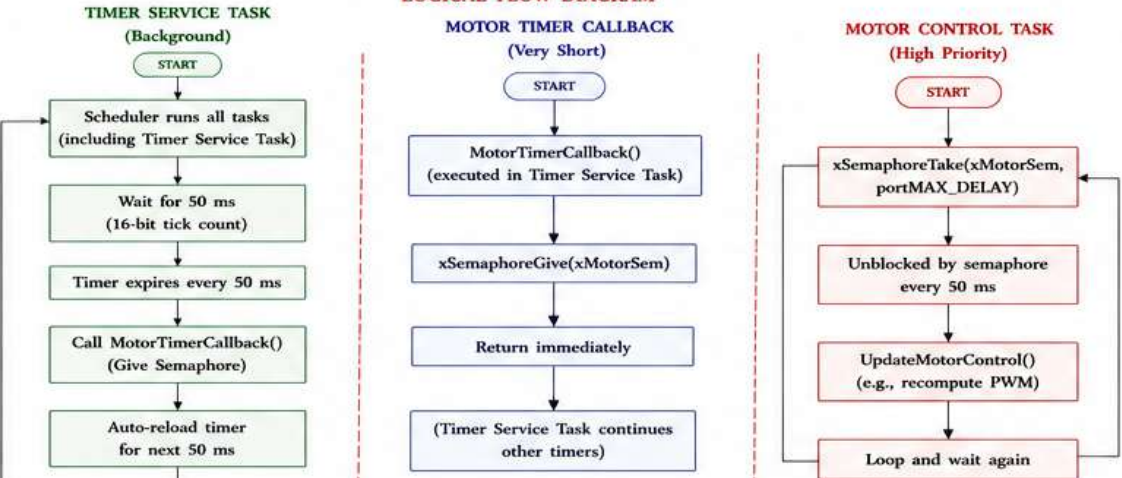
Parameter	Value
Timer Period	50 ms
Auto-reload (uxAutoReload)	pdTRUE (Periodic)
Time Base	RTOS Tick (16-bit tick count)

SEMAPHORE USAGE:

Semaphore	Given By	Taken By	Purpose
xMotorSem (Binary)	MotorTimerCallback()	MotorControlTask	Wakes motor task every 50 ms

3

LOGICAL FLOW DIAGRAM



4

EXPLANATION:

A software timer is created with a 50 ms period. Every 50 ms, the Timer Service Task calls the callback, which gives a binary semaphore. The MotorControlTask (high priority) is unblocked by the semaphore and performs the motor control update, then waits again.

KEY POINTS:

- Software timer ensures accurate 50 ms periodic trigger.
- Timer callback is kept very short (non-blocking).
- Binary semaphore safely wakes the motor control task.
- MotorControlTask runs with high priority for real-time response.
- 16-bit tick count (1 ms tick) used for time base.

ASSUMPTIONS:

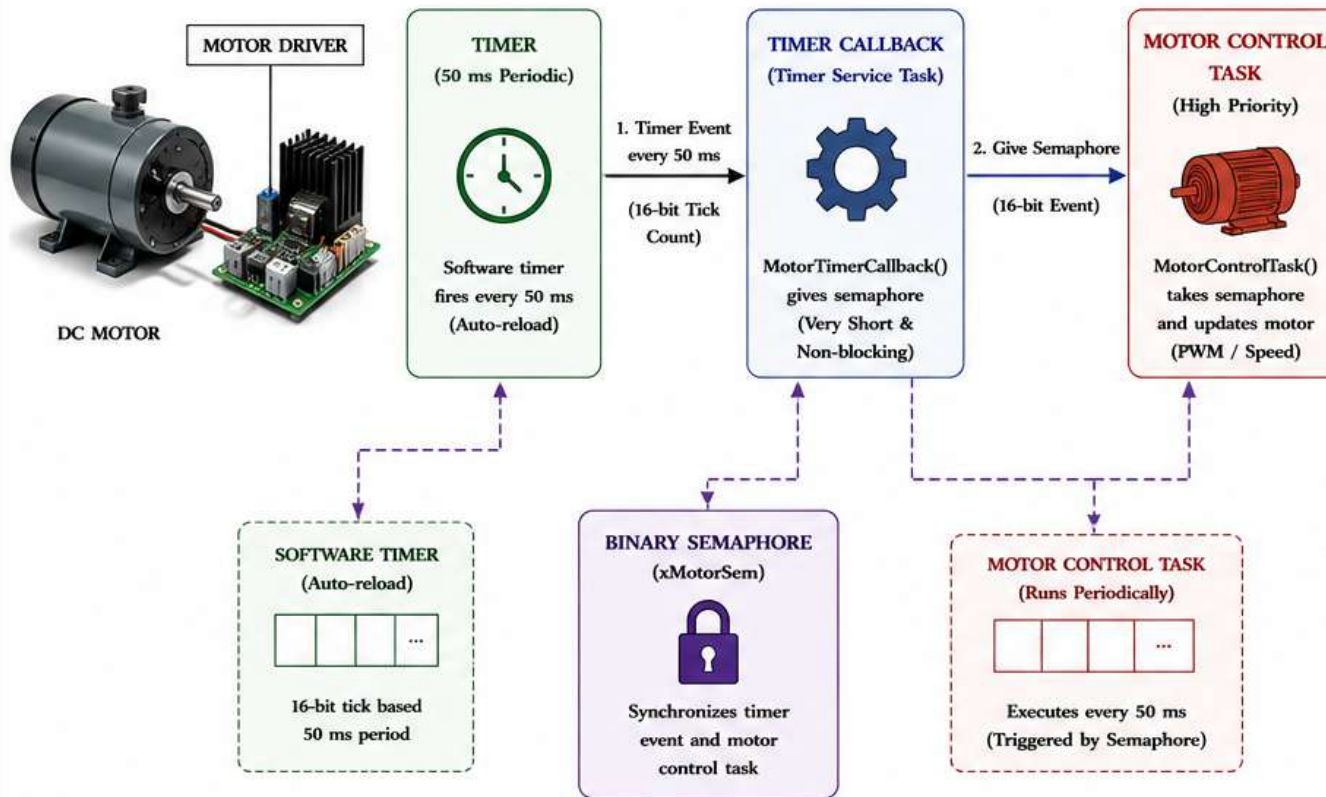
- System tick is 1 ms (16-bit tick count used).
- xMotorSem is a binary semaphore created in the given state.
- UpdateMotorControl() is fast and non-blocking.
- MotorControlTask has higher priority than the Timer Service Task.
- Timer Service Task priority is adequate to handle all timers.

5

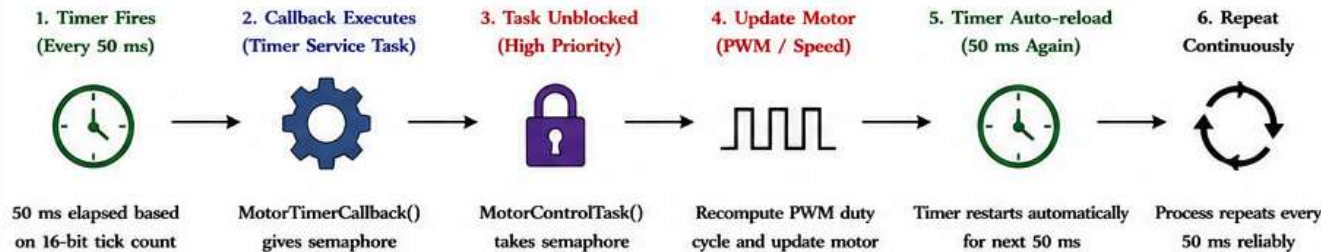
7. REAL-LIFE EXAMPLE

Example: Real-time Speed Control of a DC Motor using a 50 ms Software Timer

REAL-TIME MOTOR CONTROL SYSTEM – FLOW OF OPERATIONS



EXAMPLE FLOW IN REAL TIME



REAL-LIFE EXAMPLE:

Consider a DC motor whose speed must be regulated in real time using PWM. A software timer is used to generate a trigger every 50 ms. Each time the timer expires, it notifies the motor control task through a semaphore. The motor control task then reads the feedback (e.g., encoder), computes the new PWM duty cycle using a control algorithm (e.g., PID), and updates the motor driver.

This ensures smooth and stable motor operation with precise timing.

HOW THIS SOLVES THE PROBLEM:

- Software timer provides an accurate 50 ms periodic trigger based on 16-bit tick count.
- Semaphore safely signals the motor control task without blocking the timer service task.
- Motor control task executes with high priority, ensuring real-time response.
- Auto-reload makes the timer periodic without extra code.

BENEFITS IN REAL LIFE:

- Precise and jitter-free 50 ms interval for control updates.
- Smooth speed control and improved motor performance.
- Efficient CPU usage (no polling or busy-waiting).
- Scalable to multiple motors or tasks.

ASSUMPTIONS:

- System tick is 1 ms (16-bit tick count used).
- xMotorSem is a binary semaphore created in the given state.
- MotorControlTask has higher priority than the Timer Service Task.
- UpdateMotorControl() is fast and non-blocking.
- Timer Service Task priority is adequate to handle all timers.

QUESTION NO: 19

- Write pseudocode for a security alarm system using event flags for motion, smoke, and door sensors.

PROBLEM ANALYSIS:

The question asks for an alarm system that must respond to any one of three independent, asynchronous sensors — motion, smoke, and door.

- Each sensor is an independent event source (interrupt driven).
- The system must record which sensor triggered without losing the event.
- A single alarm monitoring task must wake up when any one (or more) of the sensors triggers.

The core problem is how multiple interrupt sources can set their own event flags safely, and how one task can wait efficiently for any of those flags without polling.

RTOS SELECTION:

FreeRTOS is selected for this solution.

- Event Groups in FreeRTOS allow multiple bits (flags) to be set by ISRs and waited by a task.
- `xEventGroupWaitBits()` with ANY condition wakes the task as soon as any sensor triggers.
- `xEventGroupSetBitsFromISR()` allows safe setting of bits from ISRs.
- Preemptive priority scheduling ensures fast response to alarms.

DATA STRUCTURE (HANDLES):

`xAlarmEventGroup : EventGroupHandle_t` (Event flags group)

EVENT FLAGS (BITS):

Flag Name	Bit	Sensor	Description
MOTION_BIT	(1 < 0)	Motion Sensor	Set when motion detected
SMOKE_BIT	(1 < 1)	Smoke Sensor	Set when smoke detected
DOOR_BIT	(1 < 2)	Door Sensor	Set when door opened
ALL_SENSOR_BITS = MOTION_BIT SMOKE_BIT DOOR_BIT			

SYSTEM OVERVIEW:



PSEUDOCODE (Contd...)

```
// Global handles (created during system initialization)
EventGroupHandle_t xAlarmEventGroup;
#define MOTION_BIT (1 < 0)
#define SMOKE_BIT (1 < 1)
#define DOOR_BIT (1 < 2)
#define ALL_SENSOR_BITS (MOTION_BIT | SMOKE_BIT | DOOR_BIT)

ISR MotionSensorISR()
BEGIN
    xEventGroupSetBitsFromISR(xAlarmEventGroup, MOTION_BIT,
    &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
END ISR

ISR SmokeSensorISR()
BEGIN
    xEventGroupSetBitsFromISR(xAlarmEventGroup, SMOKE_BIT,
    &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
END ISR

ISR DoorSensorISR()
BEGIN
    xEventGroupSetBitsFromISR(xAlarmEventGroup, DOOR_BIT,
    &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
END ISR

TASK AlarmMonitorTask(void *pvParameters) // priority = High
BEGIN
    LOOP FOREVER
        // Scheduler blocks this task until ANY one sensor bit is set
        uxBits = xEventGroupWaitBits(xAlarmEventGroup,
        ALL_SENSOR_BITS,
        pdTRUE, // clear bits on exit
        pdFALSE, // wait for ANY bit (OR)
        portMAX_DELAY);
        IF (uxBits & MOTION_BIT) THEN TriggerAlarm("Motion detected"); END IF
        IF (uxBits & SMOKE_BIT) THEN TriggerAlarm("Smoke detected"); END IF
        IF (uxBits & DOOR_BIT) THEN TriggerAlarm("Door opened"); END IF
    END LOOP
END TASK
```

2

PSEUDOCODE (Contd...)

INITIALIZATION CODE:

```
FUNCTION SystemInit()
BEGIN
    xAlarmEventGroup = xEventGroupCreate();
    CreateTask(AlarmMonitorTask, "AlarmTask", STACK_SIZE,
    NULL, PRIORITY_HIGH, NULL);
    // ISRs are assumed to be registered with hardware here
END FUNCTION
```

EVENT GROUP DETAILS:

Parameter	Value
Event Group Handle	<code>xAlarmEventGroup</code>
Wait Condition	ANY bit set (OR condition)
Clear on Exit	<code>pdTRUE</code> (bits cleared after read)
Wait Timeout	<code>portMAX_DELAY</code> (wait forever)

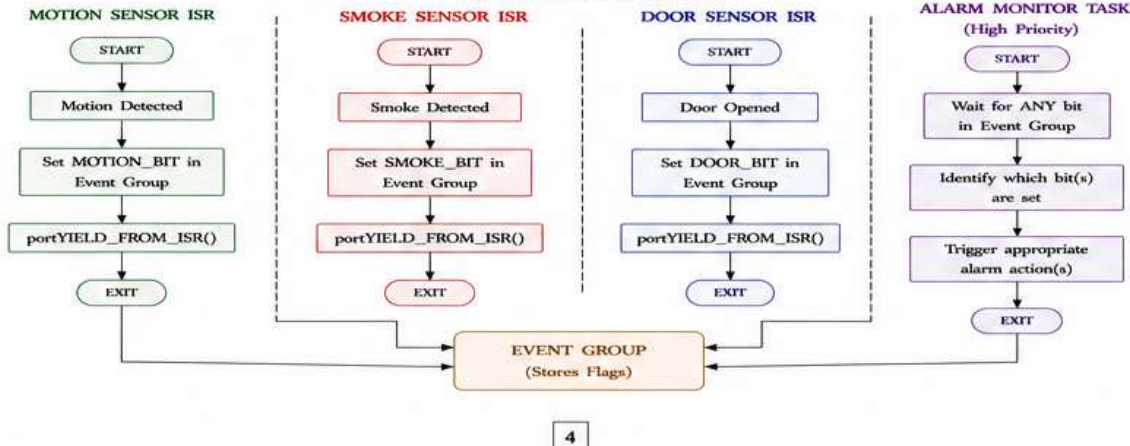
ISR API USED:

API	Purpose
<code>xEventGroupSetBitsFromISR()</code>	Set bits in event group from ISR
<code>portYIELD_FROM_ISR()</code>	Request immediate context switch if higher priority task is ready

TASK PRIORITIES:

Task	Priority Level
AlarmMonitorTask	High (Handles alarm immediately)
Other Tasks	Lower (Background operations)

LOGICAL FLOW DIAGRAM



EXPLANATION:

Each sensor has its own ISR that sets a unique bit in the event group when triggered. The AlarmMonitorTask waits for any of these bits using `xEventGroupWaitBits()` with ANY condition. When a bit is set, the task wakes up, identifies which sensor triggered, and performs the corresponding alarm action.

KEY POINTS:

- Event Groups allow multiple ISRs to set flags independently.
- ANY condition ensures the task wakes up as soon as any sensor triggers.
- Clear-on-exit resets the flags automatically after processing.
- Preemptive scheduler ensures fast response to alarms.

ASSUMPTIONS:

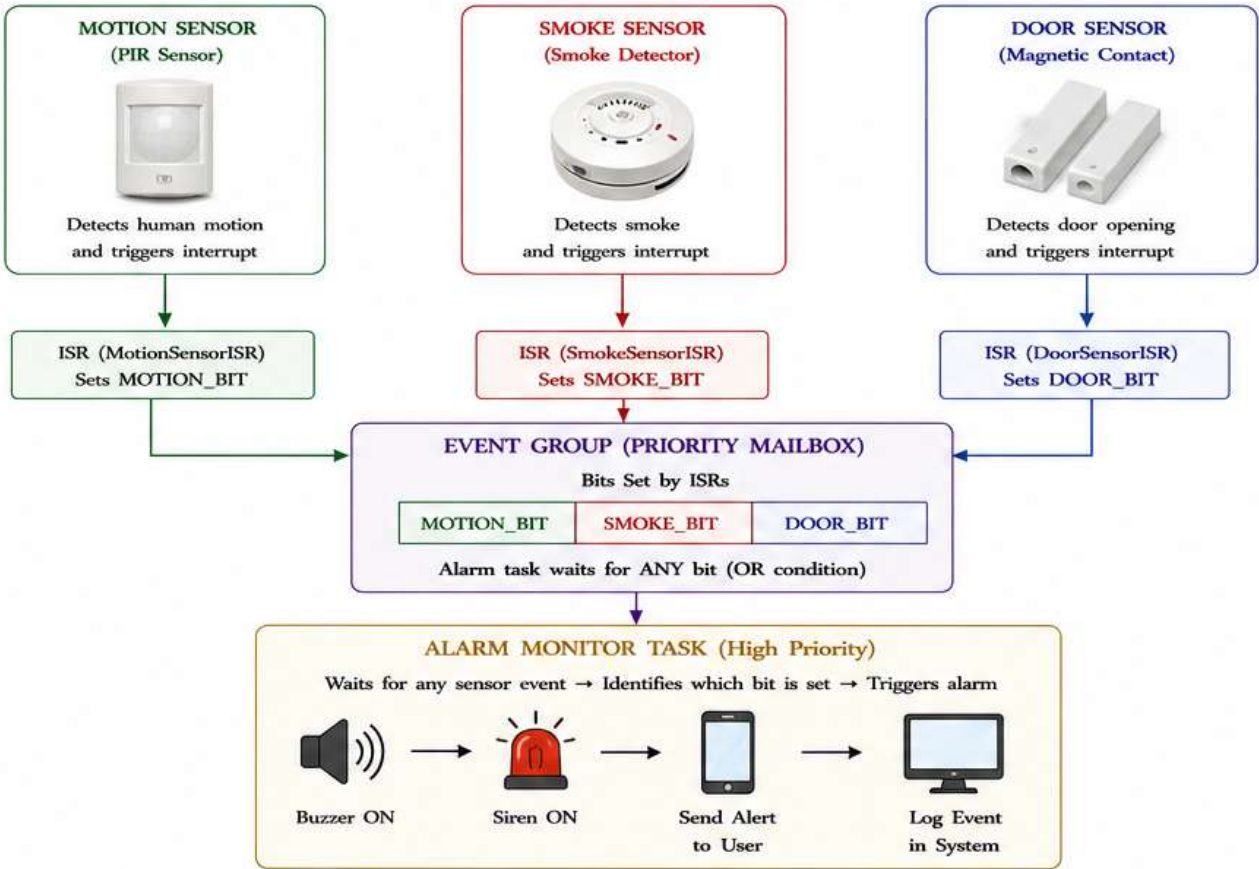
- System tick is 1 ms (16-bit tick count used).
- Event group is created at system initialization.
- Each ISR is short, fast, and non-blocking.
- Alarm actions (buzzer, siren, logging) are non-blocking or handled by other tasks.
- AlarmMonitorTask has higher priority than other tasks.

5

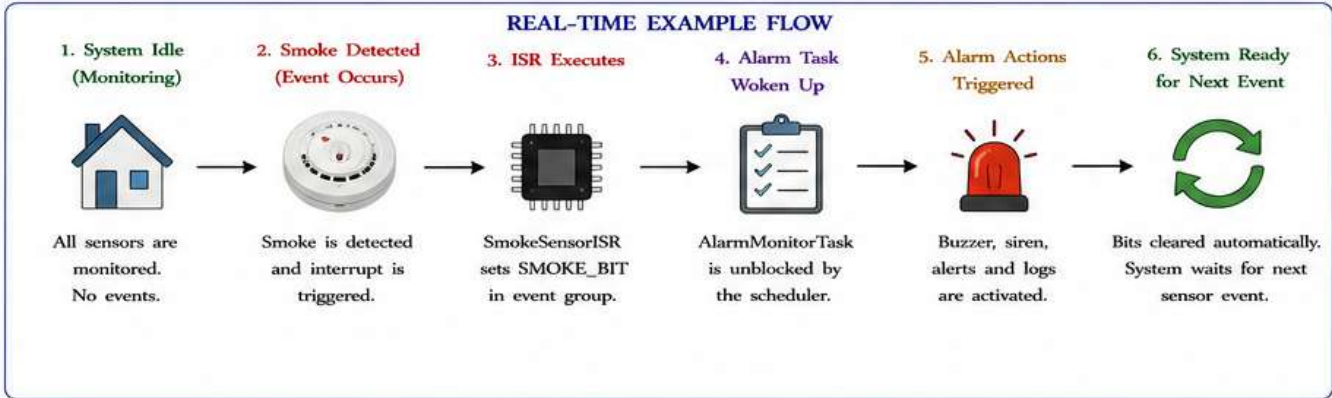
7. REAL-LIFE EXAMPLE

Example: Home Security Alarm System using Motion, Smoke and Door Sensors

REAL-TIME HOME SECURITY ALARM SYSTEM – FLOW OF OPERATION



REAL-TIME EXAMPLE FLOW



REAL-LIFE EXAMPLE:

Consider a home security system with three sensors: motion (PIR), smoke detector, and door sensor. Any sensor event should immediately trigger the alarm.

Each sensor is connected to a microcontroller and configured to generate an interrupt when triggered.

The interrupts set their respective bits in an event group.

A high-priority alarm monitor task waits for any bit to be set and performs the appropriate security actions.

HOW THIS SOLVES THE PROBLEM:

- Event flags allow multiple independent events to be handled efficiently.
- System responds immediately to any sensor event.
- No CPU time is wasted polling the sensors.
- Clear-on-exit ensures the system is ready for the next event.

BENEFITS IN REAL LIFE:

- Fast response to intrusions, fire, or door breaches.
- Improved safety and reliable monitoring.
- Can handle multiple simultaneous events.
- Scalable for more sensors in future.

ASSUMPTIONS:

- System tick is 1 ms (16-bit tick count used).
- Event group is created at system initialization.
- ISRs are short, fast and non-blocking.
- Each sensor generates one interrupt per event.
- AlarmMonitorTask has higher priority than other tasks.
- Buzzer, siren and communication functions are non-blocking.
- Event group bits are cleared automatically after processing.

EVENT BITS USED:

Bit Name	Value	Sensor	Description
MOTION_BIT	0x01 (1 << 0)	Motion Sensor	Set when motion detected
SMOKE_BIT	0x02 (1 << 1)	Smoke Sensor	Set when smoke detected
DOOR_BIT	0x04 (1 << 2)	Door Sensor	Set when door opened

ALARM ACTIONS PERFORMED:

Action	Description
Buzzer ON	Sound local buzzer to alert people nearby.
Siren ON	Activate high-power siren to deter intruders.
Send Alert	Send notification to user via mobile/app.
Log Event	Record event with timestamp in system memory.